UNIVERSIDAD DE CARABOBO

FACULTAD DE CIENCIAS Y TECNOLOGIA

FUNDAMENTOS DE PROGRAMACIÓN

No se que titulo poner jajasalu2

Participantes:

Rances Colotto 30.997.818

Ismael Armada 31.390.408

Sección 8

Valencia, 3 Abril de 2026

Descripción del problema

El problema consiste en simular la operación de un sistema de transporte marítimo con múltiples ferrys que conectan una isla con el continente. Se debe modelar la llegada de vehículos a lo largo del día, su asignación a colas según el tipo de ferry que solicitan (Express o Tradicional), y su posterior embarque respetando las capacidades máximas de cada embarcación (vehículos, pasajeros, peso) y un mínimo de vehículos requerido para zarpar. La simulación debe ser discreta, avanzando minuto a minuto, e incluir la prioridad de vehículos de emergencia.

Contexto y propósito del software desarrollado

El software desarrollado sirve como una herramienta de simulación para analizar el flujo de vehículos en un terminal de ferrys. Su propósito es evaluar la eficiencia del proceso de embarque, calcular ingresos generados por pasajes y vehículos, y determinar los tiempos de espera de los usuarios. Esto permite a los operadores del sistema tomar decisiones informadas sobre la asignación de recursos, como la cantidad de ferrys o sus capacidades, para optimizar la operación.

Análisis de la solución

Cómo se organizan y almacenan los datos

Los datos se organizan en tres categorías principales:

1. **Datos de entrada (estáticos):** Se leen desde un archivo proy1.in. La primera línea define el orden de embarque de los tres ferrys. Las líneas siguientes contienen la información de cada vehículo (código, número de pasajeros, peso, hora de llegada, placa, etc.).

2. **Datos en tiempo de ejecución (dinámicos):** Se almacenan en estructuras en memoria principal. Cada vehículo se guarda en un arreglo Vehiculo lista[]. Durante la simulación, los índices de los vehículos esperando se almacenan en dos colas circulares (ColaVehiculo) según el tipo de ferry solicitado.

Datos de configuración de ferrys: Las características de cada ferry (capacidad, tiempos, nombre) se definen estáticamente en la función inicializar_ferrys() y se almacenan en un arreglo Ferry flota[].

Justificación de las decisiones tomadas

- **Simulación basada en eventos discretos por tiempo:** Se eligió un bucle principal que avanza minuto a minuto (minuto_actual++). Esto simplifica la lógica de manejo de tiempos y es fácil de implementar y depurar para un problema con granularidad de minutos.

- **Uso de índices en lugar de copiar estructuras:** Las colas almacenan índices (int idx[MAX_VEHICULOS]) que apuntan a la posición de cada vehículo en el arreglo global lista. Esto evita la duplicación de datos y reduce el uso de memoria, ya que la información de cada vehículo se mantiene en un solo lugar.

- **Modularización:** El código se dividió en archivos (ferry_sim.h, colas.c, lectura.h, main.c) para separar la definición de estructuras (header), la lógica de colas, la lectura de datos y la lógica principal de simulación. Esto mejora la legibilidad, el mantenimiento y la reutilización del código

Definición de los structs utilizados

Definición de los structs utilizados

1. Vehiculo: Almacena toda la información relevante de un vehículo que llega al terminal.

```
typedef struct {
    int cod;        // Código del vehículo
    int npa;        // Pasajeros adultos
    int npt;        // Pasajeros tercera edad
    int tpa;        // Tipo pasaje adulto (0 primera/VIP, 1 turista/ejecutiva)
    int tpt;        // Tipo pasaje tercera edad
    int peso;       // Peso (kg o toneladas)
    int hora;       // Hora de llegada (ej: 800)
    char placa[16]; // Placa
    int tipo_ferry; // 0 Express, 1 Tradicional
} Vehiculo;
```

2. Ferry: Define las características y el estado actual de cada ferry durante la simulación.

```
typedef struct {
    char nombre[32];
    FerryTipo tipo;
    int max_vehiculos;     // Capacidad máxima de vehículos
    int min_vehiculos;     // Mínimo requerido para zarpar (30% de max)
    int max_pasajeros;     // Capacidad máxima total de pasajeros
    int max_vip_1ra;       // Capacidad máxima de asientos VIP/1ra clase
    int max_ejecutiva;     // Capacidad máxima de asientos ejecutiva/turista
    int max_peso_ton;      // Capacidad máxima de peso en toneladas
    int t_ciclo;           // Duración del viaje (ida+vuelta) en minutos
    int vehiculos[MAX_VEHICULOS]; // Índices de vehículos a bordo
    int num_vehiculos;     // Cantidad actual de vehículos a bordo
    int pasajeros;         // Pasajeros totales a bordo
    int pasajeros_tercera_edad;
    float peso_cargado;    // Peso total cargado en toneladas
    int estado;            // 1=cargando, 2=viaje, 3=espera
    int tiempo_restante;   // Tiempo restante para completar estado actual
    int id;                // Identificador (1,2,3)
} Ferry;
```

3. ColaVehiculo: Implementa una cola circular para manejar los vehículos en espera.

```
typedef struct {
    int idx[MAX_VEHICULOS]; // Arreglo de índices de vehículos
    int head, tail, size;   // Punteros de inicio, fin y tamaño actual
} ColaVehiculo;
```

Arreglos u otras estructuras aplicadas

- Vehiculo lista[MAX_VEHICULOS]: Arreglo fijo que almacena todos los vehículos leídos. Su tamaño máximo está definido por MAX_VEHICULOS.

- Ferry flota[MAX_FERRYS]: Arreglo de tamaño fijo (3) que almacena la información y estado de cada ferry.

- ColaVehiculo cola_express, cola_tradicional: Dos instancias de la cola circular para organizar los vehículos según el tipo de servicio que requieren.

Diagrama

```
[Archivo proy1.in] --> Lectura (lectura.h)
                        |
                        v
                [Vehiculo lista[]]
                        |
                        | (Asignación por hora)
                        v
    [cola_express] <---/   \---> [cola_tradicional]
          |                              |
          | (Embarque)                   | (Embarque)
          v                              v
    [Ferry flota[0]]          [Ferry flota[1]] [Ferry flota[2]]
    (Lilia C. - Express)      (La Isabela - Trad.) (La Margariteña - Trad.)
```